



DEPARTMENT OF ENGINEERING CYBERNETICS

TTK4145 - REAL-TIME PROGRAMMING

Progress Report

Authors:

Henning Kutschera Sund
henninsu@stud.ntnu.no

Øystein Revheim Martinsen
oysteirm@stud.ntnu.no

Havan Palo Darbandi
havanpd@stud.ntnu.no

Date: January 30, 2026

1 Outline

Consensus Mechanism: Our system will use a peer to peer network arrangement. To ensure consistent decision-making, the data used by each elevator must be synchronized. This is achieved by using a cyclic counter with barriers for requests, only confirming and deleting requests with consensus (figure 3), and using elevator states that have achieved consensus. This means that the button-light-contract is given when the cyclic counter for a request is at confirmed.

System Modules: In exercise one¹ we developed a working single elevator (figure 2). Together with the modules: elevator input/output, network, message synchronization and request assigner we see how they work together (figure 1). A request needs to be confirmed in the message synchronizer through messaging with the network until consensus is achieved. Then we light the button and assign it to an elevator with the assigner algorithm.

Network Protocol Choice: UDP is chosen as the network protocol primarily due to its support for low-latency communication. Assuming sufficient network bandwidth, this protocol fits our process-oriented design. By using UDP broadcast, each elevator can efficiently share its local state with all other elevators without the complexity and overhead associated with connection-oriented protocols. Any packet loss is tolerated and handled at the application level through repeated broadcasts and consensus mechanisms.

Programming Language: For this project, we will use the Go programming language. Its native support for concurrency fits well with a distributed real-time system. Goroutines and channels allow independent components to run concurrently, while the language's simplicity reduces implementation complexity and the risk of concurrency-related bugs.

2 Fault Tolerance

Description of how our system remains operational in the presence of anticipated faults. It is fundamentally based on redundancy: critical state is replicated across the system, enabling it to handle individual failures.

1. Network Loss: Detected through missing periodic state messages. Disconnected elevators continuously serving all requests. Upon reconnection, an elevator will resynchronize through the network and re-enter normal operation.
2. No Lost Orders: A request is only confirmed once all elevators currently connected to the network have signed it. This is achieved using a cyclic counter. The button lights are activated only after redundancy has been ensured.
3. Software Crash or restart: Single-process crashes are handled with process pairs, as we did in exercise four². After restart, cab calls are restored from the network based on the elevator's ID, and it can once again participate in execution of hall requests.
4. Network Packet Loss: Handled by using UDP broadcast periodically with idempotent, state-based messages. Newer messages replace older ones, allowing lost packets to be tolerated.
5. Network partition: Assigning requests locally within each partition. By prioritizing availability and partition tolerance, in accordance with the CAP theorem, the system constrains consistency, allowing multiple elevators to fulfill the same request in case of failures.
6. Motor/power loss: Each elevator has a watchdog timer that ticks when the elevator is not idle. Restart the timer each time we do something. If the timer runs out, we mark ourselves as incapable, making another elevator handle the request.

¹<https://github.com/TTK4145/Exercise3>

²<https://github.com/TTK4145/Exercise4>

Appendix

A Module Diagram

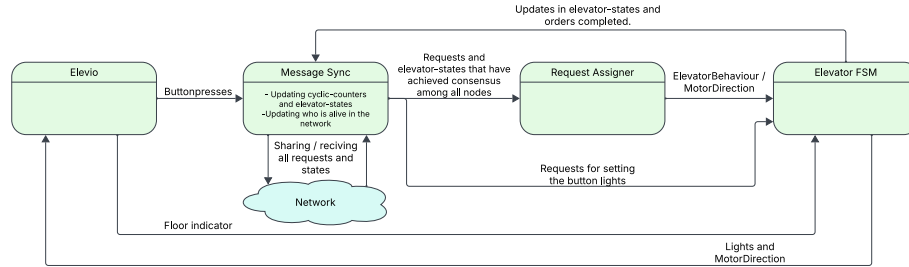


Figure 1: Flow of input and output from modules.

B State Diagram

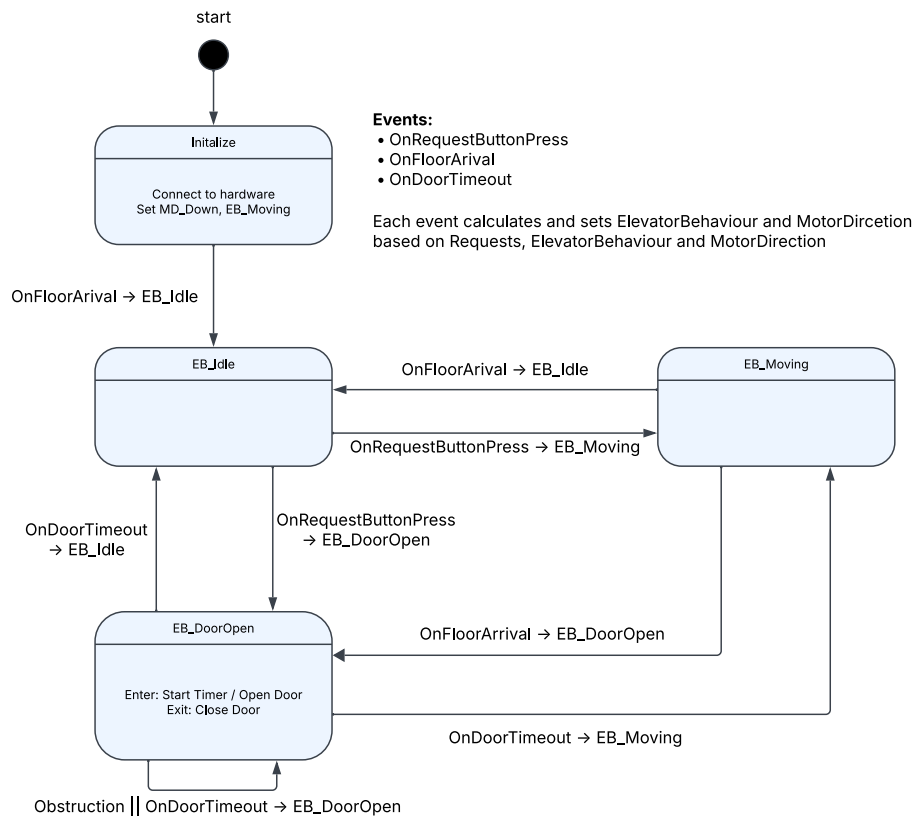


Figure 2: Overview of the single elevator state machine.

C Cyclic Counter Diagram

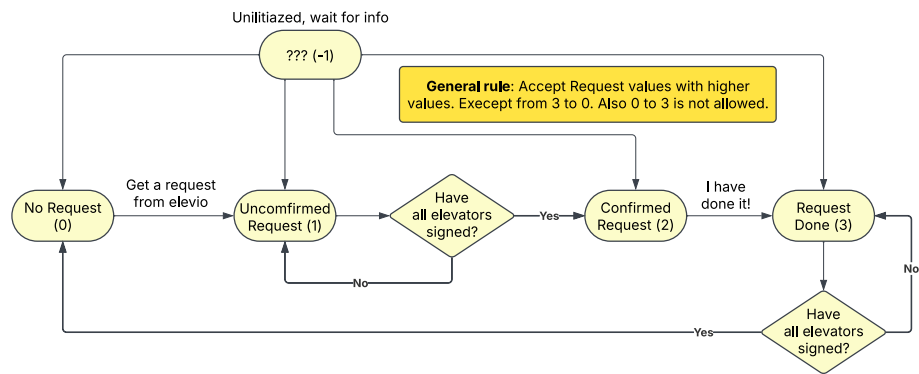


Figure 3: Request handling and assignment flow in the distributed system.